SPIKE Prime Robotics Camp

Instructor Lesson Plan — eleven sessions, eleven machines

Tony Chu

Kavosh AI & Robotics Academy · SPIKE Prime · Word Blocks → Control Theory

Contents

PART

I

How to Run This Camp

Section 1

What this plan is

Each chapter of this plan is **one camp session built around one build-instruction PDF** from the course folder. The chapter tells you (the instructor) what the machine is, what it secretly teaches, how to walk the kids through its program, what questions to ask, and how to stretch the fast finishers. The eleven sessions are ordered so that every session reuses a skill from the previous one and adds exactly one new big idea — from "press a button, see a face" all the way to a five-behaviour battle robot and a memory game with lists and broadcasts.

Key Point The arc of the camp is the arc of real robotics: **outputs → sensing → decisions → feedback control**. By session 11 the kids have met every ingredient of the machines they will see at competitions like FIRA RoboWorld Cup — state machines, sensor fusion, proportional control — without ever being told the words in advance.

Per session (≈2 h)

10 min — warm-up & hook
45 min — build
25 min — code walk-through
30 min — run, tune, challenge
10 min — reflect & tear-down

Table 1. Default time budget. Long builds (Robot Arm, Barrier Gate) steal from the challenge block.

Section 2

The live code viewer

Every session's program is hosted in **SPIKE PRIME+**, the web block editor built for this camp:

https://onemetrictony.github.io/spike-camp/editor/

(The site home, https://onemetrictony.github.io/spike-camp/, also hosts these notes and a one-click grid of every lesson.) Open any chapter's **Live Code link** (or pick the lesson in the right-hand sidebar) and the program *rebuilds itself in front of the class*, block by block — far better on a projector than a static screenshot, because the kids see the *order* in which a program grows. The viewer also:

- shows the same program as **Python** (</> Code button) — your bridge for kids who ask "what does this look like in real code?";

- **uploads to a real hub over Bluetooth** from Chrome/Edge — Connect BT with the hub's Bluetooth button blinking, then Upload (run now);
- lets kids drag, edit, and re-arrange the blocks — it is a real editor, so "what happens if we swap these two blocks?" can be answered in seconds, live.

The programs come from the actual project files (decompiled block-by-block, not retyped), so what the viewer shows *is* the code the machines run.

Section 3

Kit and prep checklist

- One SPIKE Prime set (45678) per 2–3 kids; the expansion set is needed only for Sumo and helps for Barrier Gate.
- Charged hubs (check the night before; a dead hub costs 20 minutes).
- The build PDF for the session, printed or on tablets — plus the page numbers called out in each chapter.
- Coloured bricks / black tape / printed colour cards for the colour-sensor sessions (Guitar, Music Maker, Robot Arm I, Line Follower).
- A projector with this plan's Live Code link already open.

Section 4

The control-theory thread

Every session ends with a two-minute **Control Corner**: one observation about how the machine *corrects itself* (or fails to). These accumulate into the Line Follower session (Session 9), where bang-bang → P → PID is taught explicitly, and pay off in Sumo. The thread, session by session, is noted in a box at the end of each chapter. Don't skip these — they are 2 minutes each and they are the difference between "we built toys" and "we learned robotics".

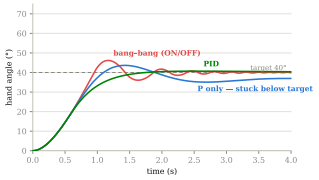


Figure 1. The camp's north star: bang-bang vs. P vs. PID response. Revisit it a little each session.

PART II

Session 1: Rock Paper Scissors

Section 5

What we're building

The simplest build of the camp — essentially the hub itself plus a handle — and deliberately so: session 1 is about the *programming environment*, not the mechanics. The hub becomes a Rock–Paper–Scissors opponent: shake it, and it rolls a secret 1–2–3, shows rock, paper or scissors on the 5×5 display, and colours the centre button. Left button starts a game with a 3–2–1 coin-sound countdown; right button plays again.

At a glance
Build: hub-only + shake handle
Source: Packt book ch. 1 (code p. 17)
New ideas: events, random, My Blocks
Sensors: gesture (shake)
Live code: `?lesson=rock_paper_scissors`

Section 6

Learning objectives

1. Navigate the SPIKE app / camp viewer; download to the hub.
2. Read an event hat ("when left button pressed") as "a stack that waits for its moment".
3. Use pick random 1 to 3 and understand why the hub never cheats.
4. Meet **My Blocks** (subroutines): start game, RPS, play again — named chunks of program you can call.

Section 7

The program

Walk it in this order (same order the viewer drags it in):

1. when program starts → write Left : the hub literally tells you which button to press. *UI in 1 block.*
2. when left button pressed → call start game : the button does nothing but delegate. All the work lives in the My Block.
3. define start game: three play sound Coin + countdown images, then call RPS . Ask: *why put the countdown in its own block?* (So play-again can skip straight to RPS.)
4. define RPS: wait until shaken → set Rps_random to pick random 1 to 3 → three if s map 1/2/3 to rock/paper/scissors image + centre-button colour.
5. define play again: sheep-sound Easter egg, reset display.

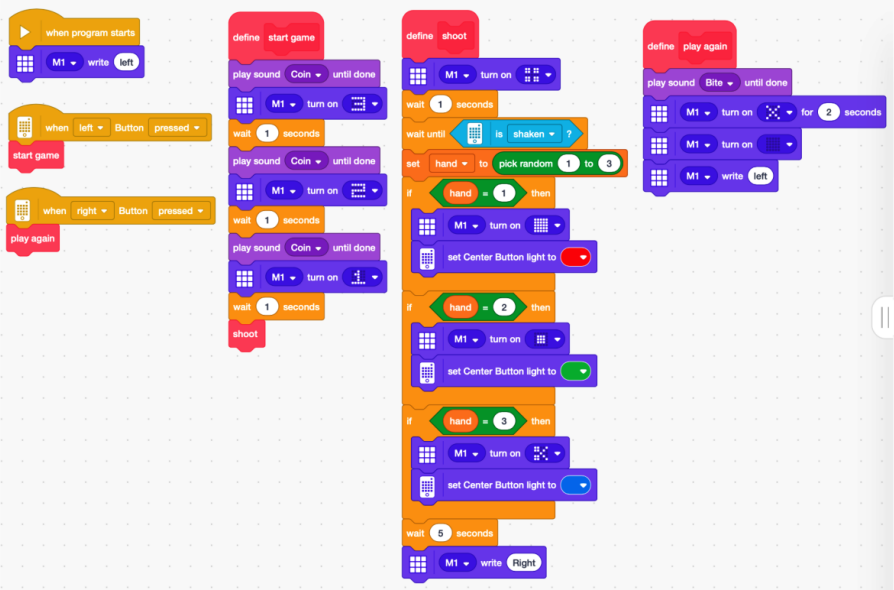


Figure 2. The full program: six stacks — two button hats, three My Block definitions, one start-up stack.

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=rock_paper_scissors

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 8

Guided questions

- *Where is the random number decided — when you shake, or when you look?* (At the shake: the `set` block runs once, then the ifs just reveal it.)
- *What would happen with* `pick random 1 to 2` *?* (Never scissors — try it live in the viewer.)
- *Why three separate* `if`*s and not if/else-if?* (Word blocks have no else-if; exactly one of the three fires because the variable can't change between them.)

Section 9

Challenges

- **Everyone:** change the sounds and draw your own rock/paper/scissors pixel art.
- **Stretch:** add a 4th option (lizard? Spock?) — forces editing the random range *and* adding an if. Most kids forget one of the two; that bug is the lesson.
- **Star:** keep score of wins vs. a human on paper — is the hub's random fair over 20 games? (Tally chart; first taste of statistics.)

Key Point

Control Corner #1: this machine makes a decision but never *checks* anything afterwards — it is pure **open loop**. Every session from here adds a little more "looking".

PART

III

Session 2: Guitar

Section 10

What we're building

At a glance

Build: neck + colour-fret guitar
Source: Packt book ch. 3 (code p. 65)
New ideas: when-condition hats, parallelism
Sensors: colour
Live code: `?lesson=guitar`

A playable LEGO guitar: the "frets" are coloured plates along the neck, and a colour sensor in the bridge reads which fret your finger covers. Each colour triggers its own note (instrument 5 = electric guitar) and a matching image. The wow factor is high and the program is tiny — ideal for cementing session 1's event idea in a new costume: **five identical stacks, five colours, five notes** (C 60, D 62, E 65/66, F 65, G 67).

Section 11

Learning objectives

1. Read a when is color hat: a stack that fires *by itself* whenever the condition becomes true — no shake, no button.
2. See that **five stacks run in parallel**: the hub watches all five colours at once. This is the camp's first concurrency conversation.
3. Map inputs to outputs through a *table* (colour → MIDI note) — the mental model behind every lookup in computing.

Section 12

The program

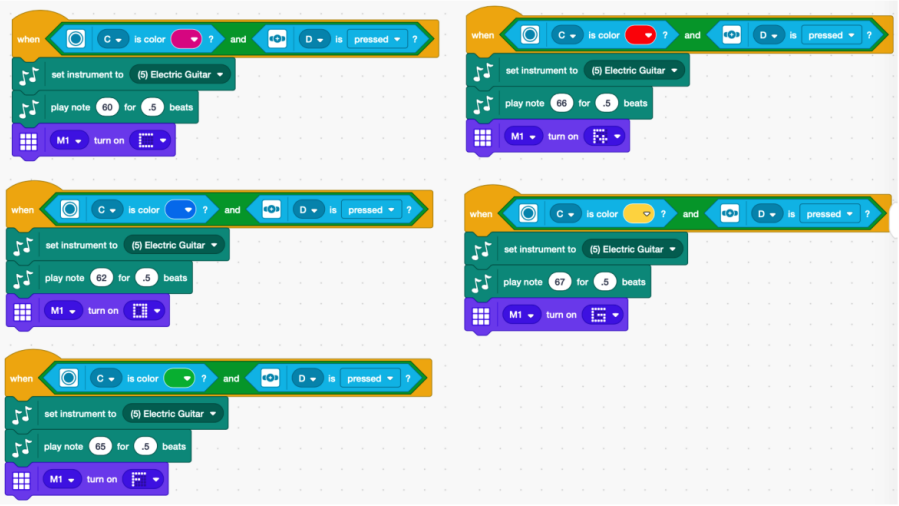


Figure 3. Five sibling stacks. Same shape, different constants — pattern recognition on a silver platter.

Show two stacks side by side and ask the class to "spot the difference" — they will find the colour and the note number in seconds. Then the punchline: *a program can be a pattern applied five times.* Introduce MIDI note numbers with the piano diagram (60 = middle C; +2 is a whole step). Let them predict note 69 before playing it (A).

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

`https://onemetrictony.github.io/spike-camp/editor/?lesson=guitar`

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 13

Guided questions

- *Why does each stack re-run* `set instrument` *?* (Any stack can fire first; each must assume nothing. Robustness through repetition.)

- *What if two fingers cover two frets?* (The sensor sees one colour — whichever is under the sensor spot. There's only one sensor: hardware limits, not code limits.)

- *Could we do this with one stack and five ifs in a forever loop?* (Yes — have a fast group build it in the viewer and compare responsiveness.)

Section 14

Challenges

- **Everyone:** retune the guitar to a pentatonic scale (60, 62, 64, 67, 69) — suddenly everything they strum sounds good.

- **Stretch:** add a sixth colour for a chord: two `play beep` blocks back to back — discover they play in sequence, not together, and discuss why.

- **Star:** use `play note ... for .25 beats` vs `.5 beats` on different frets — rhythm from geometry.

Key Point

Control Corner #2: the guitar *reacts* to the world (closed the loop by a crack!) but still never checks the *result* of its own action. Sensing input ≠ feedback — feedback is sensing your own *output*.

PART IV

Session 3: Music Maker

Section 15

What we're building

At a glance
Build: brick-track step sequencer
Source: `music-maker.pdf` (19 steps, build only!)
New ideas: programs-as-data, reconstruction
Sensors: colour
Live code: `?lesson=music_maker`

A mechanical music box: coloured 2×4 bricks sit in a rack of stations; a motor walks a colour sensor along the row; each colour plays its note. The row of bricks *is* the melody — rearranging bricks is programming.

Instructor heads-up: this PDF contains *build instructions only* — no code page. That is not a bug in the session; it *is* the session: after two sessions of reading programs, the class now **writes one from the machine's shape**. The Live Code link below is our reconstruction (labelled as such in the viewer); build the class's version together first, then compare.

Section 16

Learning objectives

1. Derive requirements from hardware: one motor, one sensor, five colours ⇒ what must the program do?
2. Reuse session 2's colour→note table inside *one* `forever` loop instead of five hats — same behaviour, different architecture.
3. Meet **tempo**: `set tempo 100 bpm` + beats = the machine's clock. The motor's speed and the tempo must agree — two clocks, one song.

Section 17

The program (reconstructed)

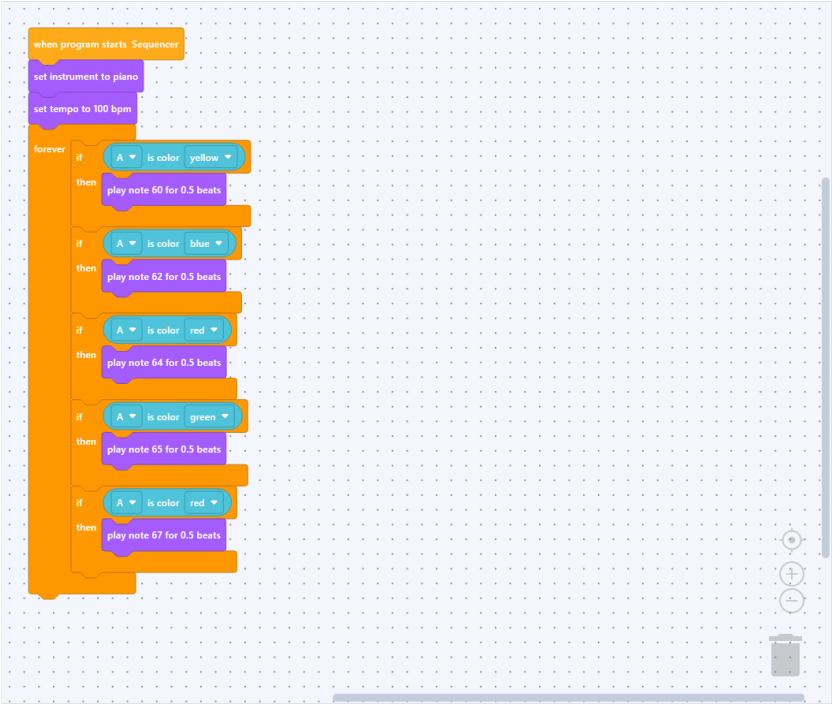


Figure 4. Our reconstruction, as rendered by the camp viewer: one start-up, one forever loop, five colour-ifs mapping to notes C D E F G.

Build it live with the class before revealing this: start from "what does the machine need to check, and how often?" Answer: *always* (forever loop), *what colour is under me now* (five ifs), *play the mapped note* (.5 beats ≈ one station of travel at 15% speed — tune together).

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=music_maker

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 18

Guided questions

- *The sensor sits between bricks — what does it see?* ("none"/black gap = a musical **rest**. The gaps are part of the notation!)
- *Guitar used five stacks; this uses one loop. When is each better?* (Hats = react anytime; loop = scan on your own schedule. A sequencer *is* a schedule.)
- *What plays if two adjacent bricks are the same colour?* (One long note or two? Try it — depends on the gap. Great edge-case discussion.)

Section 19

Challenges

- **Everyone:** compose an 8-brick melody; perform for the room.
- **Stretch:** double the tempo but keep the motor speed — what breaks? (Notes overlap stations: the two clocks disagree.)
- **Star:** add a "repeat brick": one colour reserved to mean "play the row again" — requires a variable. Seeds session 10.

Key Point

Control Corner #3: the motor drags the sensor at "about the right speed", open loop. Ask: how would the machine *know* it's on a station? (Mark stations with a detectable feature = add feedback. Park the thought.)

PART V

Session 4: Bird

Section 20

What we're building

A bird that sleeps until you reach toward it — then its ultrasonic "eyes" light up, it flaps its wings (motor F swinging 330°↔30°) and scoots forward on wheels CD. The distance sensor's four LED quadrants are programmable — the eyes literally blink.

At a glance
Build: flapping bird w/ distance sensor
Source: Packt book ch. 4 (code p. 98)
New ideas: distance events, motor sync, race conditions
Sensors: distance (ultrasonic)
Live code: `?lesson=bird`

Section 21

Learning objectives

1. Use the when closer than 5 inches hat — a *threshold* on an analogue quantity (distance), unlike colour's discrete categories.
2. Calibrate motors to zero positions at start-up (go shortest to position 0 ×3) — "home first, then act".
3. **Two stacks share one trigger**: flapping and driving both fire on the same "closer than 5 in". What if they fight over a motor? — the camp's first **race condition** conversation.

Section 22

The program

Trace the start-up stack first: eyes on, speeds low (15% drive, 10% wing — grace, not speed), all three motors homed. Then the two twin hats: one flaps in a repeat until loop with eye-blinks, one sets wheels CD and drives forward 5 rotations. They run *at the same time* — the bird flaps as it scoots. Ask what happens if both stacks had used motor F.

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

`https://onemetrictony.github.io/spike-camp/editor/?lesson=bird`

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 23

Guided questions

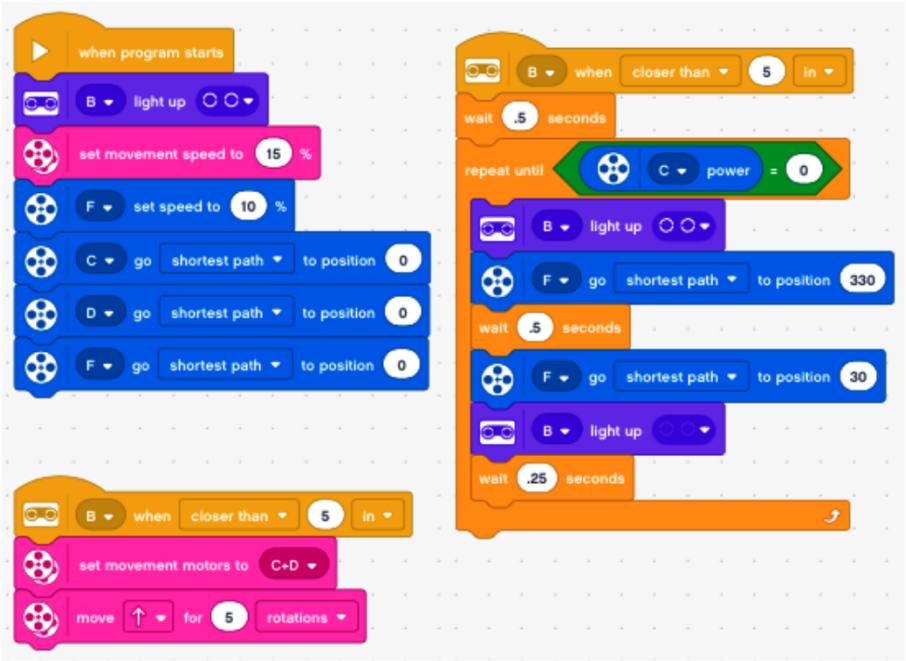


Figure 5. Start-up homing stack + two distance-triggered stacks (flap loop; drive forward).

- *Why home the motors at start instead of trusting how it was left?* (Someone always moves the wings in transport. Never trust yesterday's state — initialise.)
- *The flap goes to 330° then 30° — why not 0 to 360?* (Same point! 330→30 crosses 0 the short way: 60° of flap. Positions are a circle, not a line.)
- *Inches or centimetres — does the hub care?* (No, but your teammate reading the code does. Units are for humans.)

Section 24

Challenges

- **Everyone:** give the bird a voice — add a chirp (which stack should own the speaker? Discuss, then decide).
- **Stretch:** make it shy: approach fast and it drives *away* (second distance hat with a nearer threshold).

- **Star:** flap speed proportional to how close the hand is — needs the distance reporter inside set speed. This is **proportional control**, four sessions early. Let them find it.

Key Point

Control Corner #4: the flap loop repeats *until a motor-power condition* — the program watches a motor's own state. First time an output is also an input. That's the shape of feedback.

PART

VI

Session 5: Barrier Gate

At a glance

Build: parking barrier (68-page PDF)
Source: `barrier_gate.pdf` (code p. 66)
New ideas: if/else, wait-until sequencing, go-to-position
Sensors: distance + colour
Live code: `?lesson=barrier_gate`

Section 25

What we're building

A real-world machine: a parking-lot barrier with a ticket check. A car (any object) pulls up to the distance sensor; the gate beeps, scans a "ticket" with the colour sensor; **red ticket = valid** (lights, "Connect" jingle, bus-door sound, gate swings 355°→80°, waits for the car to pass, closes), anything else = rejected (angry double-beep, gate stays down). One long stack — the camp's first *process*.

Section 26

Learning objectives

1. Read a long sequential protocol: idle → detect → scan → decide (if/else) → act → reset. Real systems are checklists.
2. wait until as the glue between world events ("car within 15 cm", "car gone past 20 cm") — and why *two different thresholds* (15 in, 20 out) prevent stutter. This is **hysteresis**, name it!
3. go shortest to position for absolute gate angles — the gate never drifts because every move is to an *absolute* target.

Section 27

The program

Have the class act it out as a human machine first (one kid = distance sensor, one = colour scanner, one = gate arm), then map each role onto

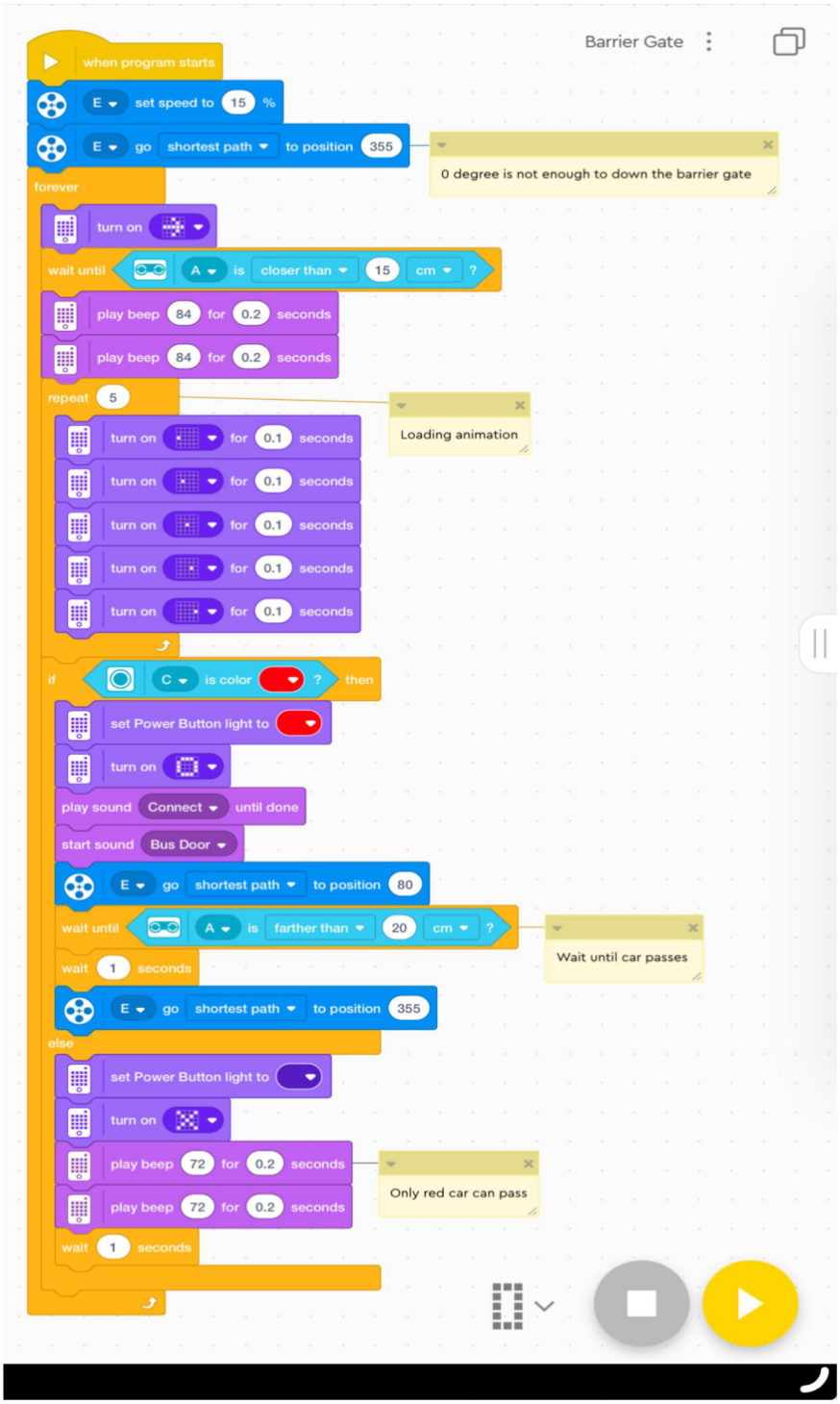


Figure 6. One stack, one process: the whole barrier protocol top to bottom, with a five-frame radar animation in a `repeat 5` loop.

the blocks. Note the 5-frame moving-dot animation (five 5×5 images, 0.1 s each, repeated 5×) — a for-loop as a film strip.

Live Code Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=barrier_gate

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 28

Guided questions

- *Why does the gate open to 80° but home to 355° (not 0)?* (355 leans the arm against its stop — mechanical preload beats software precision. Sometimes the build fixes what code can't.)
- *What if a car stops half-through the gate?* (wait until distance > 20 never fires — gate stays open. Is that a bug or a safety feature? Real barriers choose exactly this.)
- *Why check the ticket with colour and not distance?* (Different questions need different senses — match the sensor to the quantity.)

Section 29

Challenges

- **Everyone:** change the valid ticket colour; make a two-colour VIP lane (green = open longer).
- **Stretch:** add a car counter (change count by 1 + write it on the display) — parking lots are databases.
- **Star:** close the gate *early* if the car clears fast: replace wait 1 seconds with a distance condition. Compare throughput.

Key Point **Control Corner #5:** the two thresholds (15 cm to trigger, 20 cm to release) are **hysteresis** — the cure for a machine that can't make up its mind at a boundary. You will meet it again in every thermostat you ever own.

PART VII

Session 6: Robot Arm I — Manipulator Arm

At a glance

Build: 2-axis arm + gripper (53-page PDF)
Source: `Robot_Arm.pdf` (code p. 52)
New ideas: teleoperation, concurrency for real, force sensor
Sensors: colour (command deck) + force (trigger)
Live code: `?lesson=robot_arm_1`

Section 30

What we're building

The flagship build: a manipulator arm with shoulder (motor C), elbow/turret (motor E) and a gripper hand (motor A), driven by a **colour command deck** — hold coloured cards under sensor F to steer (red/yellow = shoulder up/down, blue/green = turret left/right, black = all stop) — and a **force-sensor trigger** (D) to close/open the hand. Three stacks run concurrently: a when-hat beeper, the "Arm Moving" scanner, and the "Hand" gripper cycle.

Section 31

Learning objectives

1. **Teleoperation**: the machine does nothing on its own — it amplifies a human. (Surgical robots, cranes, rovers.)
2. Concurrency with purpose: arm control and hand control are separate machines sharing one body — separate stacks, zero interference, because they own *disjoint* motors.
3. Slow is smooth: 5–7% motor speeds. Precision beats power.
4. The five-colour command language is an **opcode table** — the kids are speaking machine code with cards.

Section 32

The program

Teach the "Hand" stack as a story: home the grip to position 40 → wait for a press (beep-beep high) → clamp at −100% power → wait for the next press (beep low) → reopen. Note the *double* "wait until pressed / wait until released" pair — that is **edge detection**: react to the *change*, not the state, or one long press fires forever. The when-hat at the top (red *or*

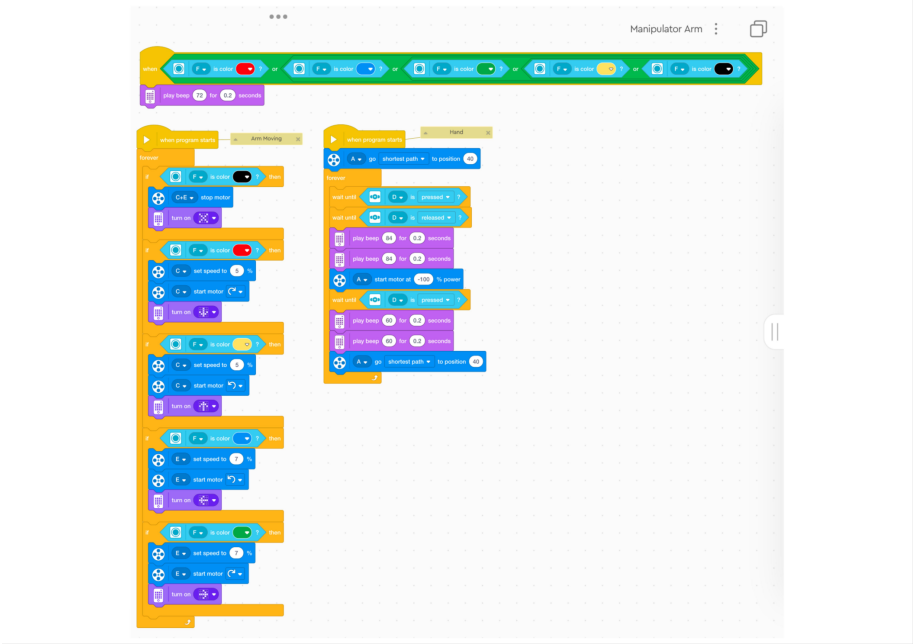


Figure 7. The exact program from the project file: when-hat beeper (top), "Arm Moving" colour scanner, "Hand" force-sensor cycle.

blue *or* green *or* yellow *or* black → beep 72) is a five-way OR chain — audible feedback that a command card was read at all.

Live Code Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=robot_arm_1

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 33

Guided questions

- *Why does clamping use* `start motor at -100% power` *but reopening uses* `go to position 40`*?* (Clamping must *stall* on the object — position control would fight to reach a pose the object forbids. Power for squeezing, position for posing.)

- *Why do C and E get different speeds (5% vs 7%)?* (Different gear trains + loads. Tuning is per-joint — ask any roboticist.)

- *What happens if you show two cards at once?* (One sensor, one reading — the ifs are checked in order each loop pass.)

Section 34

Challenges

- **Everyone:** pick-and-place relay: move three bricks across a line; time each team.
- **Stretch:** add a "dead-man's brake": no card = stop everything (currently black does this — make *absence* of any known colour do it too; harder than it sounds).
- **Star:** record-and-replay: write down the card sequence for one pick-and-place, then have another team "run" your card program. Cards = program; humans = interpreter.

Key Point **Control Corner #6:** the human IS the feedback loop — eyes measure, brain computes the error, hands issue corrections via cards. Next session we start moving that loop *into* the hub.

PART VIII

Session 7: Robot Arm II — Tilt Control

Section 35

What we're building

At a glance
Build: claw arm, hub as controller
Source: Packt book ch. 2 (code p. 46)
New ideas: IMU (yaw/pitch), deadband, tracking
Sensors: hub gyro/tilt + buttons
Live code: `?lesson=robot_arm_2`

The arm, remixed: now the **hub itself is the joystick**. Twist the hub (yaw) and the turret tracks your twist; tilt it (pitch) beyond ±3° and the shoulder raises/lowers; hub buttons nudge the claw open/closed 7° per press. Same class of machine as Session 6, radically different interface — that contrast is the lesson.

Section 36

Learning objectives

1. Meet the **IMU**: yaw = compass-like twist, pitch = nose up/down. `reset yaw` at start-up defines "zero is wherever I say".

2. **Continuous tracking**: `motor E go to position` `yaw angle` inside `forever` — the target is *live*. The motor chases a moving goal every loop tick.

3. **Deadband**: act only when |pitch| > 3°; inside ±3°, `stop`. Without it the shoulder jitters from hand tremor — let them feel the jitter by setting it to 0.

Section 37

The program

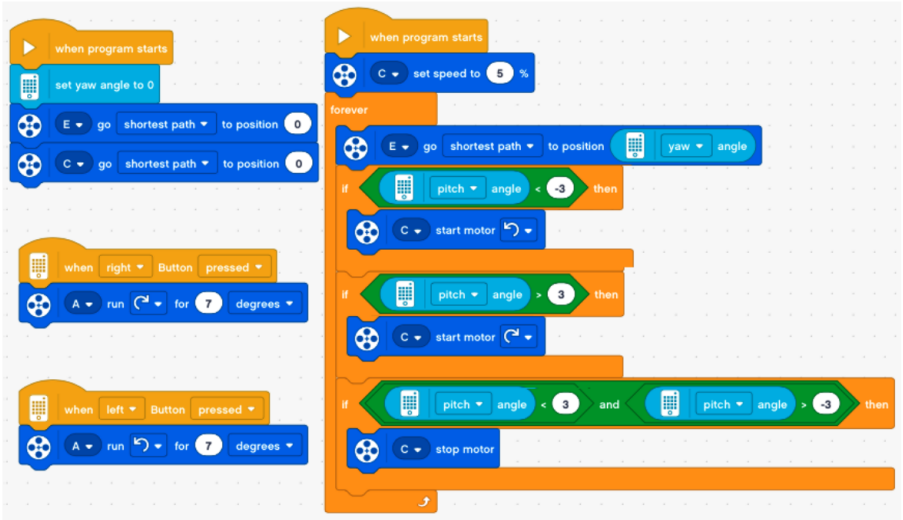


Figure 8. Homing stack, the tilt-tracking forever loop, and two button hats nudging the claw.

The heart is three ifs in the loop: pitch $< -3 \rightarrow$ shoulder down; pitch $> 3 \rightarrow$ shoulder up; $-3 <$ pitch $< 3 \rightarrow$ stop. Draw the number line on the board with the "dead" zone shaded. Then the turret line — one block doing what session 6 needed four colour-ifs for: *position tracks yaw, continuously.*

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=robot_arm_2

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 38

Guided questions

- *Why 7° per button press for the claw and not "hold to close"?* (Discrete, repeatable steps — count presses to reproduce a grip. Session 6's power-clamp vs this: two grip philosophies.)
- *What if you walk around holding the hub?* (Yaw drifts with you — the turret follows. Feature or bug? Depends what you're building. `reset yaw` is the fix — when should it run?)
- *Why does the deadband compare to ±3 and not ±0.5?* (Hands shake about that much. Match thresholds to the noise, not to wishful thinking.)

Section 39

Challenges

- **Everyone:** tune the deadband: find the smallest value that doesn't jitter for *your* hand.
- **Stretch:** invert the controls (plane-style pitch); have another kid fly it — whose muscle memory wins?
- **Star:** shoulder speed proportional to |pitch|: gentle tilt = slow, steep = fast. That's a **P controller** you just wrote.

Key Point

Control Corner #7: the deadband is a confession: near the target we'd rather do *nothing* than fight noise. P control (Session 9) replaces the confession with mathematics — respond a *proportional* amount instead.

PART IX

Session 8: Domino Machine

Section 40

What we're building

A cart that *lays a domino run* as it drives: magazine of dominoes, a cam (motor C, 640° per cycle) that plants one tile, then the cart creeps 3 cm and plants the next — ten tiles per run, then `stop all`. The project ships two variants (straight and curved run — the curved one steers with movement speed −10%); we teach the curved file.

At a glance

Build: domino-laying cart (curved track ver.)
Source: `sp-domino` PDF + 2 code PNGs
New ideas: dead reckoning, calibration UI, My Blocks II
Sensors: force (as a button)
Live code: `?lesson=domino`

Instructor note: the two My Blocks in the shipped file have *Chinese names* (the project is from a Chinese classroom). First activity: rename them to plant run and trim in the app — and discuss why names matter. The logic is untouched by renaming — that's the point.

Section 41

Learning objectives

1. **Dead reckoning**: the cart *believes* 640° of cam = one tile planted and 3 cm = one spacing — nothing checks it. Ten tiles of accumulating error: watch tile 1 vs tile 10.

2. **Calibration interface**: the trim My Block nudges the cam ±0.1 s while you hold the force button — a maintenance mode built into the product. Machines need service hatches.

3. My Blocks with a purpose: the main loop reads like a sentence: forever : call trim , call plant run .

Section 42

The program

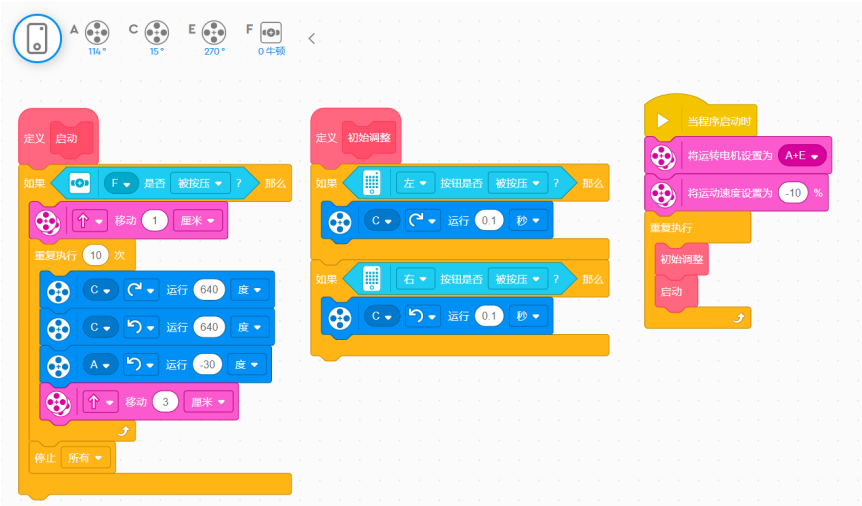


Figure 9. The shipped code image (curved variant). Two My Block definitions + the tiny main stack.

Walk the plant run definition: guard condition (force sensor as arm/disarm switch), move 1 cm lead-in, then repeat 10 { cam 640° cw, cam 640° ccw,

move 3 cm }, stop all. Ask why the cam turns forward *and back* each cycle (return stroke — the cam must re-cock).

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

`https://onemetrictony.github.io/spike-camp/editor/?lesson=domino`

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 43

Guided questions

- *Why does the curved version differ from the straight one by essentially one number?* (Steering bias. One parameter changes the artwork — parameters are power.)
- *Tile 10 lands 1 cm off. Which knob do you turn — spacing, cam angle, or speed?* (Systematic error → spacing; random error → mechanics. Diagnose before tuning!)
- *Could the cart* check *a tile was planted?* (Colour sensor looking back? Force feedback on the cam? Cost vs. confidence — engineering's eternal trade.)

Section 44

Challenges

- **Everyone:** longest successful run wins (increase the repeat count until physics says no).
- **Stretch:** S-curve: alternate steering sign every 3 tiles.
- **Star:** two carts, one starting where the other ends — requires agreeing on a handoff protocol. (Teams of machines = FIRA thinking.)

Key Point

Control Corner #8: dead reckoning is open-loop navigation, and its error *accumulates*. Every self-driving car, vacuum robot and Mars rover is a war against exactly this. Next session, we finally close the loop.

PART X

Session 9: Line Follower — Colour & Control

At a glance

Build: simple 2-motor cart + down-facing colour sensor
Source: book's shared colour-figures PDF (theory ch.)
New ideas: feedback control — bang-bang, P, toward PID
Sensors: colour (reflectivity mode)
Live code: `?lesson=color_image`

Section 45

What we're building

The capstone of the control thread. No fancy build — any two-motor cart with a colour sensor pointing at the floor — because today the *program* is the machine. Black tape line on white floor; the cart follows the **edge** of the line. This session cashes in every Control Corner so far.

Section 46

Learning objectives

1. The line follower loop: **measure** → **compare** → **steer**, forever. Error e = target reflectivity − measured.
2. **Bang-bang**: on black steer one way, on white the other (our demo program). It works, it waddles — feel why.
3. **P control**: steering = $K_p \cdot e$. Smooth on straights, cuts corners; increase K_p until wobble — meet *oscillation*.
4. Where I and D come from (fig. in margin of Part 1): I fixes the steady lean gravity/asymmetry causes; D calms the wobble by reacting to the error's *speed*. P = now, I = past, D = future.

Section 47

The program

Run the bang-bang demo first and let everyone watch the waddle. Then derive P on the board from the waddle itself: "the cart overreacts identically whether it's barely off or way off — what if the steer amount *was* the error?" Rebuild live in the viewer: replace both fixed steers with start moving steer (K_p×(50 − reflectivity)) . Tune K_p as a class: 0.5 (lazy), 1.5 (nice), 4 (wobble monster).

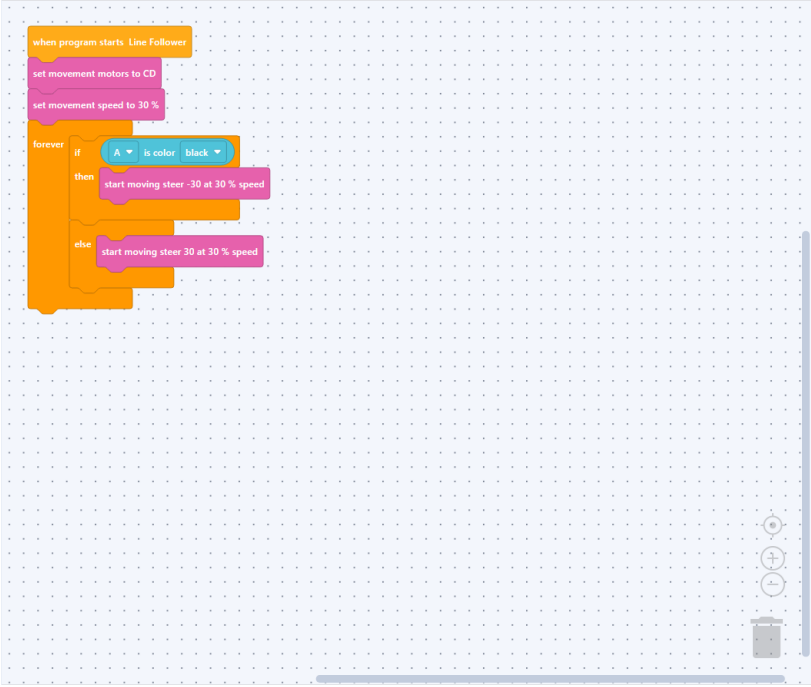


Figure 10. The bang-bang starter, as rendered by the camp viewer. The session's work is improving it.

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=color_image

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 48

Guided questions

- *Why follow the edge and not the middle of the line?* (On the line, left and right look identical — no signed error. The edge gives you a direction, not just a distance.)
- *What does reflectivity 50 mean physically?* (Half tape, half floor — straddling the edge. The setpoint is a place.)
- *Why does big K_p oscillate?* (Correction arrives late — the cart has already crossed the edge. Delay + gain = wobble; that's why D exists.)

Section 49

Challenges

- **Everyone:** tune K_p for the fastest clean lap of a taped oval.
- **Stretch:** add I: accumulate error each loop, add $K_i \times$sum to the steer. Watch it fix the constant lean on a banked/tilted board.
- **Star:** full PID with $K_d \times$(error − last error); time-trial against every other team's controller. This is, literally, competition robotics.

Key Point **Control Corner #9 (the payoff):** bang-bang, deadband, hysteresis, P, I, D — the kids have now *felt* all of them on hardware. The line follower is the "hello, world" of control theory, and they wrote it themselves.

PART XI

Session 10: Sumo Bot

At a glance
Build: battle wedge, expansion set helps
Source: Packt book ch. 5 (code p. 138)
New ideas: state machines, subsumption, competition
Sensors: distance, colour (ring edge), force (bumper)
Live code: `?lesson=sumo`

Section 50

What we're building

Robot sumo: two robots, one white-ringed black arena, push or be pushed. The shipped program is the most sophisticated architecture of the camp: a **state machine**. A variable `action` (1–5) names the current behaviour — patrol, edge-escape, chase, brace, bumper-attack — and *four concurrent sensor stacks* vote to change it while one big loop *performs* whatever the current state demands.

Section 51

Learning objectives

1. **State machine**: behaviour = a named mode; sensors change the mode; one performer loop acts the mode out. Draw the five-state diagram on the board *before* reading a single block.
2. **Priority (subsumption)**: seeing the white ring edge overrides chasing; chasing overrides patrol. Survival > aggression > boredom.
3. Randomness as strategy: after an edge-escape the bot turns a `random` amount — predictable robots lose.

Section 52

The program

Read the performer loop as a menu: action 1 = reverse patrol at −25%; 2 = lunge forward 2 rotations then random turn (edge recovery); 3 = meow, then charge −75% while the target is near; 4 = brace/push; 5 = bumper-triggered attack. Then the watchdogs: reflectivity high (white ring!) → action 2; distance < 6 → action 3; force bumper → action 5; combinations → action 4. One variable is the robot's entire mind.

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

`https://onemetrictony.github.io/spike-camp/editor/?lesson=sumo`

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 53

Guided questions

- *Why does the ring edge set a state instead of steering directly?* (So the *reaction* lives in one place. Watchdogs sense; the performer acts. Separation of concerns.)
- *Two watchdogs fire in the same instant — who wins?* (Last write wins. Is that ordering guaranteed? No! Great debate; real robots use explicit priorities.)
- *Why patrol* backwards*?* (The wedge is the weapon and it's on the back. Strategy lives in the build, code follows the body.)

Section 54

Challenges

- **Everyone:** tournament! Round-robin, two-minute bouts.
- **Stretch:** tune the personality: aggressive (chase threshold 8 cm) vs cautious (edge check more often) — name your fighter.
- **Star:** add a 6th state: "victory dance" when the opponent is gone for 5 s. Requires a timer pattern — sneaky-hard with blocks.

Key Point **Control Corner #10:** the chase state uses proportional pursuit thinking (closer ⇒ act differently) and the whole robot is a feedback system where the "sensor" includes *another robot trying to beat you*. Control in an adversarial world = the entire discipline of competition robotics.

PART XII

Session 11: Simon Says

At a glance
Build: hub + external 3x3 matrix, tilt input
Source: Packt book ch. 7 (code p. 184)
New ideas: lists, broadcasts, debugging a real shipped bug
Sensors: hub IMU (tilt as 4-way joystick)
Live code: `?lesson=simon_says`

Section 55

What we're building

The grand finale: a full memory game — 12 stacks, 115 blocks, every idea of the camp in one program. The hub plays a growing colour/tone sequence (broadcasts 1–4 trigger four "show colour" stacks); the player answers by *tilting* the hub (roll/pitch read as a 4-way joystick); a **list** (colors to remember) stores the sequence; wrong answer → game-over jingle and your score. Intro and game-over music are real multi-note tunes.

Section 56

Learning objectives

1. **Lists**: the sequence lives in colors to remember ; add , item n of , length of — a growing memory.
2. **Broadcasts**: broadcast item count ... and wait → the matching when I receive n stack plays that colour. Messages decouple "what to say" from "how to show it".
3. **Debugging as a lesson**: the shipped project file contains a *real bug* — it appends the constant 0 to the list instead of pick random 1 to 4 (the book's screenshot has the fix). Load the file, watch the game demand the same answer forever, and let the class *find and fix it*. Nothing you can teach beats fixing a real shipped bug.

Section 57

The program

Teach the main loop as three phases per round: **grow** (append to the list), **show** (loop over the list, broadcasting each item and waiting), **test** (for

each item: arm the tilt readers via `direction show`, wait for `color selected`, compare with `item count of list`; mismatch → `broadcast game over`). The four tilt-hat stacks each just beep and set `color selected` — tiny ISRs, if you want the fancy word.

Live Code

Open this session's program in the camp block editor — the blocks drag themselves in, one by one, exactly as the kids should build them:

https://onemetrictony.github.io/spike-camp/editor/?lesson=simon_says

From the viewer you can toggle </> Code to show the equivalent Python, Export .py, or connect a hub over Bluetooth (Connect BT → Upload) and run it live — no SPIKE app needed.

Section 58

Guided questions

- *Why broadcasts instead of `call`?* (`and wait` gives sequencing; and four receivers can share one message vocabulary. Also: My Blocks can't be triggered from "show" AND "test" phases as cleanly.)
- *Why does the bug (always 0) make the game unwinnable rather than crash?* (0 is a *valid-looking* value — the worst bugs are legal values with illegal meaning.)
- *Where does the difficulty curve come from?* (Nowhere explicit — the list's growth IS the difficulty. Emergent design.)

Section 59

Challenges

- **Everyone:** fix the bug; then camp high-score board.
- **Stretch:** speed up playback as the list grows (`set tempo` scaled by `length of list`).
- **Star:** two-player duel: alternate who answers, two score variables. Requires re-architecting the test phase — proper software engineering.

Key Point

Control Corner #11 (closing): eleven machines ago, a button made a picture. Today the kids debugged a 115-block concurrent, message-passing, list-driven game — and they can tell you where its state lives, what its inputs are, and how it fails. That's not "playing with LEGO".

That's systems engineering.

PART XIII

Appendix: Session–Skill Matrix

Section 60

What each session banks

#	Session	Banked skill
1	Rock Paper Scissors	events, random, My Blocks
2	Guitar	condition hats, parallel stacks
3	Music Maker	programs-as-data, loops vs hats, tempo
4	Bird	thresholds, homing, race conditions
5	Barrier Gate	if/else protocols, hysteresis
6	Robot Arm I	teleoperation, concurrency, edge detection
7	Robot Arm II	IMU, live tracking, deadband
8	Domino	dead reckoning, calibration, parameters
9	Line Follower	feedback: bang-bang → P → PID
10	Sumo	state machines, priorities, strategy
11	Simon Says	lists, broadcasts, debugging

Section 61

All live-code links

- Site home (editor + these notes): https://onemetrictony.github.io/spike-camp/
- Editor: https://onemetrictony.github.io/spike-camp/editor/
- Append `?lesson=` + one of: `rock_paper_scissors`, `guitar`, `music_maker`, `bird`, `barrier_gate`, `robot_arm_1`, `robot_arm_2`, `domino`, `color_image`, `sumo`, `simon_says`

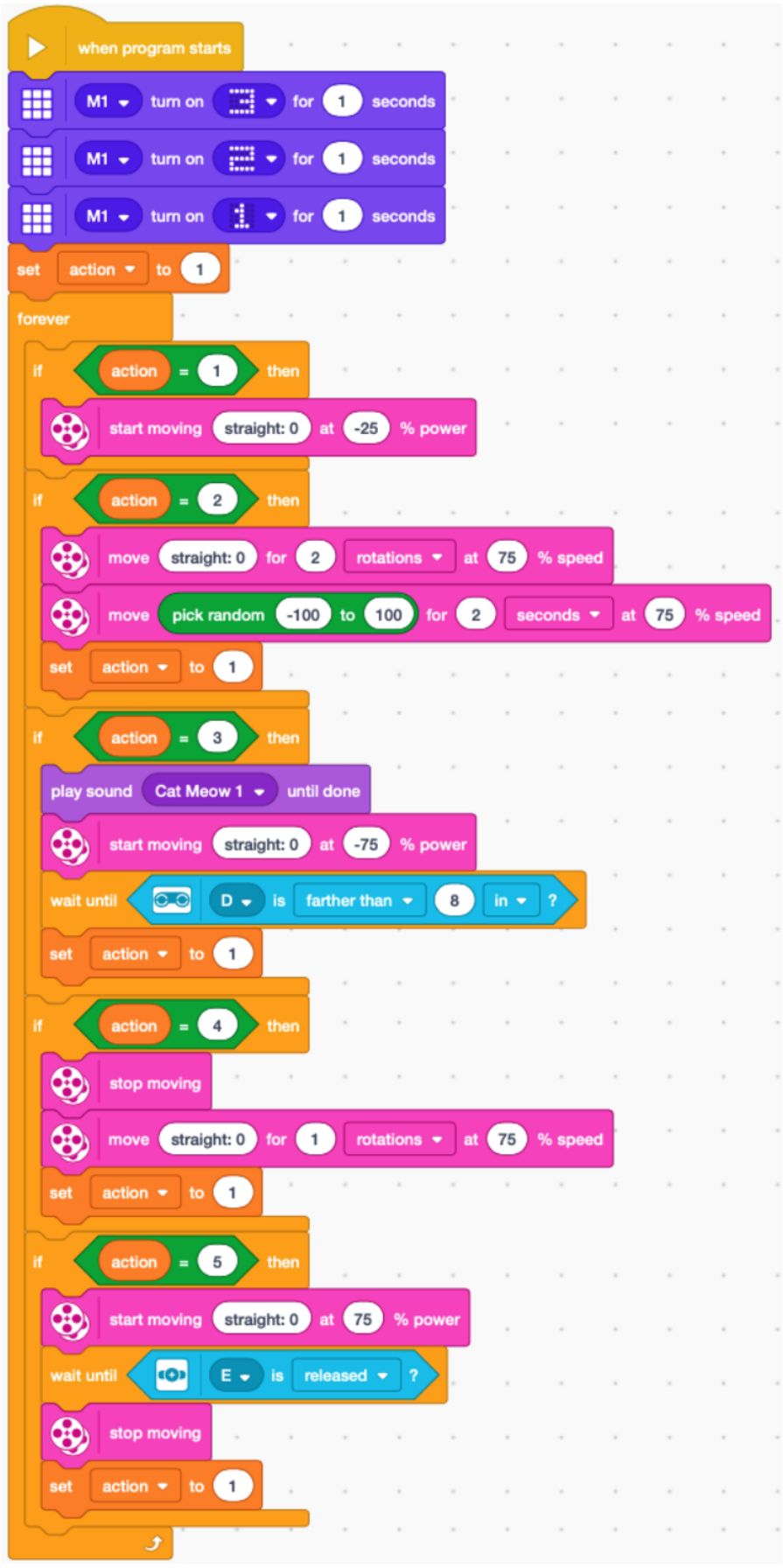
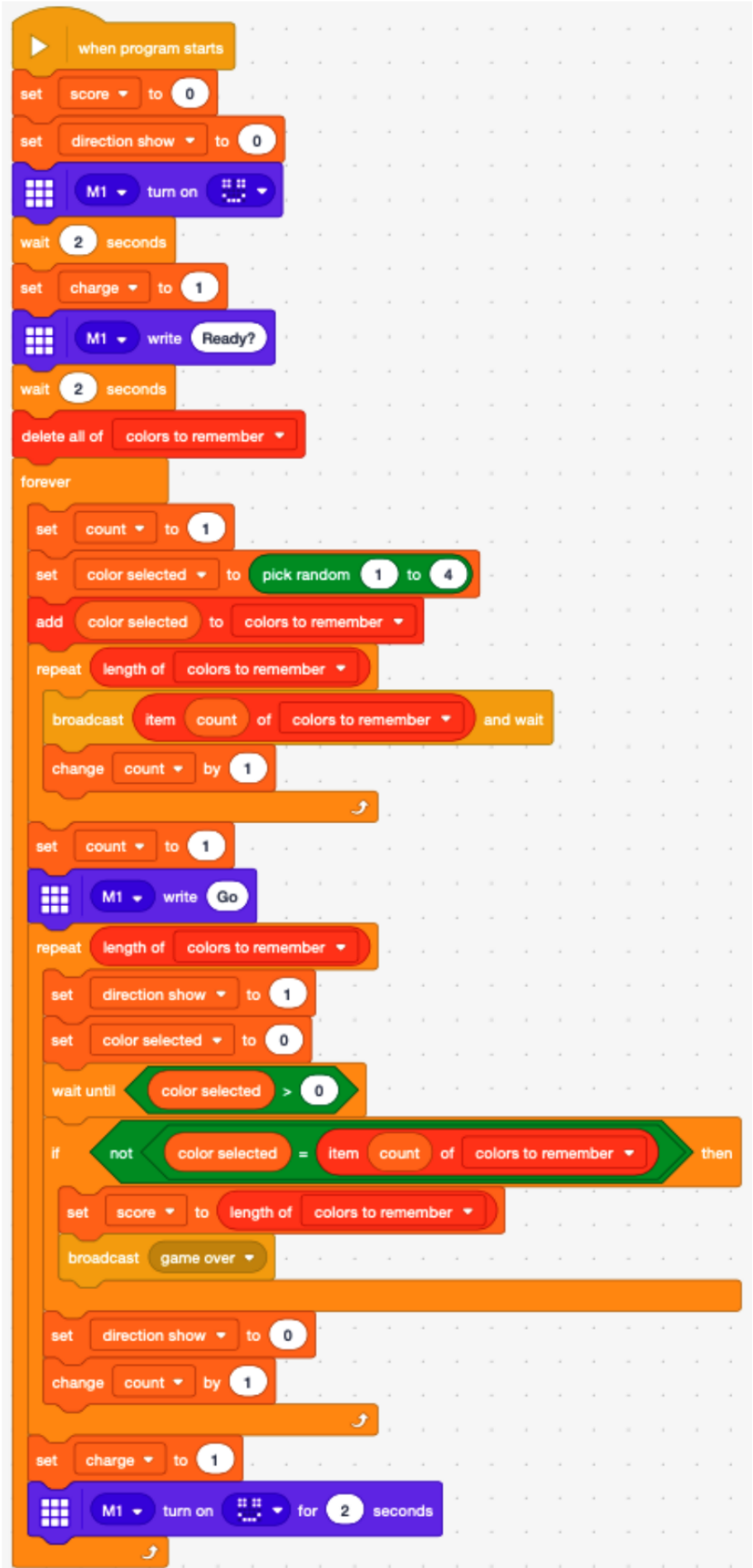


Figure 11. Six stacks: 3-2-1 countdown + performer loop; movement-pair setup; kill switch; and three watchdog stacks writing `action`.


when program starts
set score to 0
set direction show to 0
M1 turn on
wait 2 seconds
set charge to 1
M1 write Ready?
wait 2 seconds
delete all of colors to remember
forever
set count to 1
set color selected to pick random 1 to 4
add color selected to colors to remember
repeat length of colors to remember
broadcast item count of colors to remember and wait
change count by 1
set count to 1
M1 write Go
repeat length of colors to remember
set direction show to 1
set color selected to 0
wait until color selected > 0
if not color selected = item count of colors to remember then
set score to length of colors to remember
broadcast game over
set direction show to 0
change count by 1
set charge to 1
M1 turn on for 2 seconds